

CSEN 296B-2 | Week 2.2 Lab Submission

Backlog and Sprint Zero Plan for AstraNotes

Student Name: Atishay Jain | Date: April 10, 2026 | Project: AstraNotes | Technical Path: Python

1. User Stories and Acceptance Criteria

The following eight user stories are derived directly from the Requirement Set established in Week 1.2 (FR-01 through FR-08, NFR-01 through NFR-03, SPR-01 through SPR-04). Each story includes testable acceptance criteria aligned with the Definition of Done from Week 2.1. The traceability chain runs from requirements to stories to acceptance criteria to backlog to the Sprint Zero plan.

US-01 — Create a New Note

Requirement: FR-01, FR-07

As a user, I want to create a new note by providing a title and body so that I can capture my thoughts and have them saved with a unique ID, timestamps, and an empty tag list.

Acceptance Criteria:

- When I submit a valid title and body, a Note is created with a UUID, created_at, and modified_at set automatically.
- The new note is persisted to the local JSON file store immediately after creation.
- Submitting an empty title or empty body is rejected with a non-technical validation error message (ValidationError per SPR-02).
- The created note can be retrieved by its ID in the same session and after restarting the application.

US-02 — Edit an Existing Note

Requirement: FR-02

As a user, I want to edit an existing note's title or body so that I can keep my notes accurate and up to date, with the last-modified timestamp updating automatically.

Acceptance Criteria:

- When I modify and save a note, only the changed fields and the modified_at timestamp are updated; the original created_at and ID remain unchanged.
- The updated note is persisted to storage before the operation returns.
- If the target note does not exist, the system raises NoteNotFoundError and shows a user-friendly message.
- If the save operation fails, a clear error is shown and the existing note is not corrupted.

US-03 — Delete a Note

Requirement: FR-03

As a user, I want to delete a note by its unique identifier so that I can remove notes I no longer need and keep my collection organized.

Acceptance Criteria:

- When I trigger delete and confirm, the note's JSON file is removed from the storage directory.
- After deletion, the note no longer appears in list or search results.
- Attempting to delete a non-existent note raises NoteNotFoundError with a clear, non-technical message per SPR-02.

US-04 — Mark a Note as Private

Requirement: FR-04, SPR-01

As a user, I want to flag a note as private so that its body is encrypted at rest and my sensitive content is protected even if someone accesses the storage files directly.

Acceptance Criteria:

- When `is_private` is set to `True` and the note is saved, the body is encrypted by `PrivacyService` using Fernet before being written to disk.
- The raw note file on disk contains no plaintext body content for any private note.
- When a private note is retrieved, the body is decrypted by `PrivacyService` transparently so the user sees readable content.
- If encryption or decryption fails, a clear error is shown and no partial or garbled content is displayed.
- Toggling a note from private back to non-private stores the body in plaintext and removes the encrypted version.

US-05 — Persist and Restore Notes Across Sessions

Requirement: FR-05, FR-06, NFR-03

As a user, I want my notes to be saved to local files and automatically restored when I reopen the application so that I never lose data between sessions.

Acceptance Criteria:

- On startup, all previously saved notes are loaded from local file storage and displayed.
- Each note is stored as an individual `{id}.json` file in the designated storage directory.
- Notes created or edited in the current session are available in the next session without additional action.
- If a note file is missing or corrupt on startup, the app logs a `PersistenceError` and continues loading the remaining notes.
- Startup and list operations for a collection of 500 notes complete within 2 seconds per NFR-01.

US-06 — Search Notes by Keyword

Requirement: FR-08, NFR-01

As a user, I want to search my notes by keyword so that I can find relevant notes without scrolling through my entire collection.

Acceptance Criteria:

- Entering a keyword returns all notes whose title or body contains it (case-insensitive).
- Results are returned as a list of matching note summaries (ID, title, snippet).
- If no notes match, a clear empty-state message is shown.
- Search across up to 500 notes completes within 2 seconds per NFR-01.
- Private note bodies are decrypted before searching so that keyword matches are not missed.

US-07 — Handle Storage Errors Gracefully

Requirement: SPR-02, SPR-03

As a user, I want clear and non-technical error messages when something goes wrong with saving or loading my notes so that I understand what happened without seeing raw stack traces.

Acceptance Criteria:

- File-not-found, permission-denied, and corrupt-data scenarios each produce a distinct, user-friendly message.
- Domain exceptions (`NoteNotFoundError`, `PersistenceError`, `ValidationError`) are used consistently throughout the codebase.
- No raw Python tracebacks or file paths are displayed to the user.
- Every error path has either an automated test or a documented manual verification procedure per SPR-03.

US-08 — Project Setup and Structural Readiness

Requirement: NFR-02, NFR-03, SPR-04

As a developer, I want the project to be organized into distinct layers (models, services, repositories, app logic) with dependency injection support so that I can build, test, and extend each component independently.

Acceptance Criteria:

- The project directory follows the layered structure: models/, services/, repositories/, app/, tests/ per NFR-02.
- The NoteRepository abstract base class is defined with save(), get(), list_all(), update(), and delete() method signatures.
- A requirements.txt or pyproject.toml file documents all third-party dependencies with their purpose and license per SPR-04.
- A simple test can instantiate any service with a mock repository, confirming dependency injection works per NFR-03.
- The project runs with `python -m astranotes` without errors (placeholder CLI acceptable).

2. Prioritized Backlog

Stories are ordered by a dependency and risk criterion: stories that establish foundational infrastructure or de-risk the architecture are placed first. Stories that depend on a stable core follow. This ordering reflects the Working Agreement rule that I will not begin coding a component until its design has been reviewed, and the Waterfall phase gates from Lab 1.1.

Pri	Story	Requirement(s)	Rationale
1	US-08 — Project Setup and Structural Readiness	NFR-02, NFR-03, SPR-04	Foundation: all other stories depend on the layered folder structure, repository interface, and DI support.
2	US-01 — Create a New Note	FR-01, FR-07	Primary user action. Validates Note model, Validation Layer, and JSON File Adapter end-to-end.
3	US-04 — Mark a Note as Private	FR-04, SPR-01	Highest architectural risk. Privacy-Service must be verified before the save/load path is considered stable.
4	US-05 — Persist and Restore Notes	FR-05, FR-06, NFR-03	Validates the full read/write round-trip including startup loading and 500-note performance target.
5	US-02 — Edit an Existing Note	FR-02	Depends on create and persist. Validates the update path through NoteRepository.
6	US-03 — Delete a Note	FR-03	Completes CRUD. Depends on create and persist being solid.
7	US-06 — Search Notes by Keyword	FR-08, NFR-01	Depends on a populated note store and privacy decryption. Performance needs real data to validate.
8	US-07 — Handle Storage Errors Gracefully	SPR-02, SPR-03	Cross-cutting concern; easier to finalize once all storage paths are implemented.

Why US-08 first: The project structure and repository interface are prerequisites for every other story. Without the layered directory and the NoteRepository ABC, there is no contract for the JSON File Adapter, Privacy Service, or Validation Layer to implement against.

Why US-04 third: The Privacy Service is the highest architectural risk in AstraNotes. If encryption is not integrated into the save/load path early, every CRUD story built on top of the repository risks introducing plaintext storage bugs that violate SPR-01. This aligns with the Privacy and Security Gate (Criterion 5) in my Definition of Done.

Why US-07 last: Error handling is a cross-cutting concern that touches every storage path. It is easier to finalize and test once the full CRUD loop and search are implemented, rather than trying to handle all edge cases before the code paths exist.

3. Sprint Zero Plan

Sprint Duration: Week 2.2 through end of Week 3 | **Goal:** Establish the project foundation so that Sprint 1 can begin implementation with zero ambiguity about structure, tools, workflow, or architecture.

Sprint Zero is not a feature sprint. No user stories are marked Done at the end of Sprint Zero. The sprint is complete when the working environment, architecture skeleton, and planning artifacts are all in place and verified against the Definition of Done.

Objective 1: Repository and Project Structure (NFR-02, SPR-04)

- Initialize a Python project repository with folders: astranotes/ (models, repository, privacy, validation, cli), tests/, planning/, docs/.
- Add pyproject.toml listing only verified, licensed dependencies (cryptography for Fernet, pytest for testing) per SPR-04.
- Add a .gitignore appropriate for Python projects.
- Confirm the project runs with `python -m astranotes` without errors (placeholder CLI acceptable).
- **DoD check:** Structure matches architecture from Submission 1; no untracked dependencies; project runs cleanly.

Objective 2: Note Data Model (FR-01, FR-07)

- Implement the Note dataclass: id (UUID), title (str), body (str), is_private (bool), created_at (datetime), modified_at (datetime), tags (list of str).
- Write unit tests: Note instantiates with valid fields; created_at and modified_at are set automatically; id is a valid UUID.
- **DoD check:** Model is explainable, tested, and traced to FR-01 and FR-07.

Objective 3: NoteRepository Interface and JsonFileRepository (FR-05)

- Implement abstract NoteRepository with method signatures: save, get, list_all, update, delete.
- Implement JsonFileRepository that reads/writes individual .json files per note to a local data directory.
- Write unit test: a note can be saved to a temp directory and retrieved with matching fields.
- **DoD check:** Repository interface in place; save/load round trip passes; traced to FR-05.

Objective 4: PrivacyService Skeleton (FR-04, SPR-01)

- Implement PrivacyService with `encrypt(body: str) -> bytes` and `decrypt(data: bytes) -> str` using Fernet (cryptography library).
- Write unit test: encrypting a string and decrypting it returns the original value; encrypted output is not equal to plaintext.
- *Risk note:* Key management is deferred to Sprint 1. A hardcoded test key is acceptable in the test suite only.
- **DoD check:** PrivacyService tested for basic round-trip; no plaintext stored for private notes; traced to SPR-01 and FR-04.

Objective 5: ValidationLayer Skeleton (FR-01)

- Implement ValidationLayer with at least one rule: note title must be a non-empty string.
- Raise a custom ValidationError when the rule is violated.
- Write unit test: empty title raises ValidationError; valid title passes.
- **DoD check:** ValidationLayer tested and traced to FR-01.

Objective 6: Planning Artifacts and Workflow Readiness (All)

- Create planning/ directory containing requirements.md, user-stories.md, backlog.md, and sprint-zero-plan.md mirroring this submission.
- Confirm the Architecture Decision Log from Week 1.2 is accessible and up to date in docs/.
- Establish docs/prompt-log.md and record Sprint Zero decisions (dependency choices, directory layout rationale) as first entries.
- Confirm the Working Agreement and Definition of Done from Week 2.1 are applied to every Sprint Zero task.
- Run all unit tests and confirm 100% pass rate before Sprint Zero is closed.

Sprint Zero Risks

Risk	Likelihood	Mitigation
Fernet key management unclear for production use	Medium	Defer to Sprint 1; use test key only in Sprint Zero. Document in ADL.
JSON edge cases (corrupt files, missing notes on startup)	Low	Implement basic error handling in Sprint Zero. Defer advanced recovery to US-07.
Sprint Zero expands into feature work	Medium	No user story is marked Done in Sprint Zero. Feature work stays in the backlog.
Architecture needs adjustment once skeleton code is written	Low–Med	Sprint Zero skeleton is the test. Update ADL before Sprint 1 if changes needed.
Scope creep from AI suggestions or own impulses	Medium	Working Agreement Section 1.2 and 1.5 scope discipline rules apply. Lab 1.1 Cloud Sync lesson as reminder.

Sprint Zero Exit Criteria

Sprint Zero is complete only when all of the following are true:

- Project repository is initialized with the layered folder structure defined above.
- Note dataclass is implemented and has passing unit tests.
- NoteRepository interface and JsonFileRepository are implemented with a passing save/load round-trip test.
- PrivacyService is implemented with a passing encrypt/decrypt test.
- ValidationLayer is implemented with a passing title validation test.
- planning/ and docs/ folders are populated and current (prompt log, ADL, backlog, user stories).
- All tests pass with 100% pass rate.
- No user story from the backlog is prematurely marked Done.

4. How AI Helped and What I Refined, Changed, or Rejected

AI was used as a planning partner across three phases of this lab — exploration, drafting, and critique — following the three modes defined in Section 1.2 of my Working Agreement.

What AI helped with:

I asked AI to generate an initial set of user stories from my Week 1.2 requirement IDs (FR-01 through FR-08, NFR-01 through NFR-03, SPR-01 through SPR-04) and the architecture components from Submission 1. The first draft covered the core functional requirements but the stories were generic — they read like requirements restated in "As a user..." format without connecting to the specific AstraNotes architecture. I also asked AI to suggest a prioritized backlog ordering and a Sprint Zero objective list.

What I refined:

I revised every user story to reference the actual architecture components (PrivacyService, JsonFileRepository, ValidationLayer, NoteManager) and the specific domain exceptions (NoteNotFoundError, PersistenceError, ValidationError) from my Week 1.2 architecture submission. I tightened acceptance criteria to include the performance target from NFR-01 (under 2 seconds for 500 notes) and the privacy constraint from SPR-01

wherever they applied. I also changed vague criteria phrased as "the system should..." into specific, testable conditions.

What I changed:

The AI initially suggested a backlog ordering that placed search (US-06) as priority 2, reasoning that search is a high-value user feature. I reordered the backlog to place project setup (US-08) first and privacy (US-04) third, because every story depends on the repository interface being defined, and the PrivacyService boundary must be validated early to avoid the plaintext storage risk flagged in my Definition of Done (Criterion 5). The AI's ordering optimized for user value; mine optimizes for technical dependency and risk reduction, which is more appropriate for a solo developer in a waterfall-gated course.

What I rejected:

The AI suggested adding a user story for "user authentication" — entirely out of scope for a local, single-user note application. I rejected it immediately. It also proposed a Sprint Zero objective for "CI/CD pipeline setup" that is premature for a solo quarter-length project and would violate the scope discipline rules in Section 1.2 of my Working Agreement.

The AI also suggested a separate story for tag filtering and a Version History story in the initial backlog. I rejected the tag filtering story because filtering by tag is not one of my committed 15 requirements — adding it would be the same scope creep risk the Cloud Sync change request demonstrated in Lab 1.1. I deferred Version History to a later sprint because, while part of the architecture, it is not tied to the highest-priority functional requirements and building it before CRUD and privacy are solid would be premature.

The process confirmed the Working Agreement from Week 2.1: AI is useful for generating a first draft quickly, but every output required review against the requirement set, the architecture, and the project scope before it was accepted. The final artifacts reflect my judgment, not the AI's first answer. I am comfortable defending every prioritization decision and every story in this submission without additional preparation, satisfying the willingness-to-defend criterion (Criterion 9) in my Definition of Done.